

1.- Qué es el overflow y cuando puede ser considerado como una amenaza. Pon un ejemplo

El **overflow** es cuando se llena la memoria más de lo esperado. El atacante puede utilizar la vulnerabilidad para escribir caracteres basura hasta que se llene la memoria donde se almacena y después utilizar lo otro que ponga con código malicioso. Es decir, el atacante llena la memoria con basura y cuando ha superado el límite introduce algún código malicioso.

Por ejemplo, si dentro del código especificamos que el usuario puede introducir solo cinco caracteres y no lo validamos puede darse esta vulnerabilidad. El atacante escribirá cinco caracteres con "basura" y, a continuación, introducirá el código o la acción maliciosa que desea hacer.

2.-Cita y describe brevemente las vulnerabilidades que debemos evitar en lenguajes de alto nivel junto con la forma de corregirlas.

- **Inyección SQL:** permite al atacante realizar una ejecución de código SQL. Se puede evitar mediante parámetros como ? o %S para que el programa no tome lo que introduce el usuario como código directamente.

- **XSS:** permite introducir código malicioso para ejecutar en usuario válidos. Para evitarlo necesitamos sanitizar el código.

- **CSRF:** permite que se ejecute código en la página sin validar. Validar lo que se nos muestra en pantalla, y escapar los caracteres HTML.

- **Falta de autorización:** no comprobar si el usuario tiene o no autorización para poder acceder a algo. El usuario solo pone su usuario y lo que quiere hacer pero no se comprueba si tiene acceso, por ejemplo, al archivo que quiere acceder. Se soluciona poniendo una función que compruebe si el usuario tiene acceso o no a ese archivo.

- **Path traversal:** cuando se le permite al usuario modificar algo de una ruta concreta. El atacante puede escapar de esa ruta utilizando ../ o cosas parecidas. La solución está en no permitir que se realicen o se pongan esos caracteres. Evitar que el usuario pueda salir de la zona segura. Es decir, por ejemplo no permitir nombres que empiecen con ../ o /.

- **Deserialización insegura:** cuando se procesan objetos serializados que meten usuarios no válidos. Esto se solucionaría validando lo que mete el usuario para que no ponga nada malicioso.

- **Autenticación insegura:** permitir guardar los usuario sin cifrar. Es decir, que te deje meter contraseñas y las guarde en texto plano. Esto se soluciona guardando las credenciales cifradas en el código o log.

- **Paso de datos inseguro:** cuando se transmite un mensaje de información sensible sin cifrar, siendo vulnerable a todo tipo de ataques. Se puede cifrar los datos con hlib para que no vayan en texto plano.

- **DoS/DdoS:** ataques de servicio hacia un equipo en concreto. Validar todo el contenido que pone el usuario en el código para evitar que se utilicen para este tipo de ataques.

- **Uso de eval incorrecto:** eval ejecuta todo el código que proporcione el usuario sin validar. Hay que utilizar otro tipo de funciones para ejecutar código que permita la validación de lo que pone el usuario.

- **Otros tipos de inyección:** el código permite otro tipo de inyección de, por ejemplo, LAD. Como en la inyección de SQL, hay que evitar que tome como código todo lo que proporciona el usuario.

- **Inyección de código:** permite al usuario introducir código en un formato que no debería permitir código. Es decir, por ejemplo se pide el nombre y el atacante aprovecha para introducir el nombre y el código que puede ser. Como en las demás, sanitizando el código puede ayudar.

- **Falta de validación:** por ejemplo, si un programa sirve para dividir como a/b, el programa no valida si la división se puede o no hacer. Lo que podría acabar rompiendo el programa. Esto se evitaría, en este caso, validando que el usuario pone dígitos y no letras, y que en el b no ponga ceros.

- **Manejo erróneo de errores:** cuando el programa tiene un error, muestra por pantalla el tipo de error que es. Esto se soluciona poniendo un mensaje generalizado de errores sin especificar en ningún sitio el tipo de error.

3.-Indica 3 pruebas de software que creas que, para ti, son más interesantes, indicando el por qué, describiendo brevemente lo que se hace en ellas.

- **Pruebas unitarias:** estas pruebas verifican que las partes por separado del código estén correctas. Es decir, que las partes del código funcionen bien por solitario. Esto nos ayuda a ver que las partes del código estén correctas.

- **Pruebas de sistema:** comprobar que el sistema entero funcione conjuntamente. Con esto comprobamos que funcione el sistema como tal. Es decir, que funcione el propósito del programa.

- **Pruebas de programa:** esto nos ayuda para comprobar que todos los componentes del programa funcionen correctamente. Es la prueba final, comprobando que todo funcione en conjunto y cumpla con su función que se nos pide y con todo lo que debería realizar.

Ejercicio 1-python (Respuesta al final del código)

Indica dónde se encuentra o encuentran (de existir) la vulnerabilidad o vulnerabilidades en este código

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2 from urllib.parse import parse_qs

3 class SimpleHandler(BaseHTTPRequestHandler):
4     def do_GET(self):
5         query = self.path.split('?', 1)
6         params = {}
7         if len(query) > 1:
8             params = parse_qs(query[1])
9             name = params.get('name', [''])[0]

10            self.send_response(200)
11            self.send_header('Content-type', 'text/html')
12            self.end_headers()
13            respuesta = f'<html><body><h1>Hola, {name}!</h1></body></html>'
14            self.wfile.write(respuesta.encode('utf-8'))

15 if __name__ == "__main__":
16     server = HTTPServer(('localhost', 8000), SimpleHandler)
17     print("Servidor iniciado en http://localhost:8000")
18     server.serve_forever()
```

Respuesta:

El problema está en la línea de la respuesta. Pues se deja visualizar la variable “name” que ha introducido el usuario sin escapar ni validar. Esto puede generar vulnerabilidades tipo XSS o CSRF. Permitiendo ejecutar código malicioso en la víctima sin que el usuario lo sepa. Esto se puede solucionar usando el `html.escape`. Es decir, podríamos importar el módulo, creo que era `html`, y usar el parámetro `escape`. Así escapamos lo introducido por el usuario a `html`. Evitamos así que el atacante puede utilizar el código para algo maligno. Por ejemplo, podríamos añadir algo así al código:

```
name_seguro=html.escape{name}

respuesta=f'<html><body><h1>Hola, {name_seguro}!</h1></body></html>'
```

Significado del código:

1. `from http.server import BaseHTTPRequestHandler, HTTPServer`
Importa clases necesarias para crear un servidor HTTP básico y para manejar las peticiones HTTP que recibe el servidor.
2. `from urllib.parse import parse_qs`
Importa el método para analizar la cadena de consulta (query string) de la URL en un diccionario de parámetros, facilitando extraer los valores enviados en la URL.
3. `class SimpleHandler(BaseHTTPRequestHandler):`
Define una clase llamada SimpleHandler que extiende la clase base BaseHTTPRequestHandler; esta clase será la encargada de procesar las peticiones HTTP entrantes.
4. `def do_GET(self):`
Define el método que responde a las peticiones HTTP GET. Es el código que se ejecutará al recibir una solicitud GET del cliente.
5. `query = self.path.split('?', 1)`
Toma la ruta completa de la petición HTTP (self.path) y la divide en dos partes separadas por el signo ?, limitando a una división. Esto extrae la ruta básica y los parámetros de consulta.
6. `params = {}`
Inicializa un diccionario vacío para almacenar los parámetros de consulta extraídos.
7. `if len(query) > 1:`
Verifica si la petición tiene parámetros de consulta (es decir, si hay algo tras el ?).
8. `params = parse_qs(query[1])`
Si existen parámetros, los analiza usando parse_qs para convertir la cadena en un diccionario con listas de valores para cada parámetro.
9. `name = params.get('name', [''])[0]`
Obtiene el primer valor del parámetro "name" si existe; si no, asigna cadena vacía. Este es el dato introducido por el usuario.
10. `self.send_response(200)`
Envía una respuesta HTTP con el código 200 indicando éxito en la petición.
11. `self.send_header('Content-type', 'text/html')`
Añade la cabecera HTTP que indica que el contenido que se enviará será HTML.
12. `self.end_headers()`
Finaliza la sección de cabeceras HTTP, permitiendo empezar a enviar el cuerpo del mensaje.
13. `respuesta = f"<html><body><h1>Hola, {name}</h1></body></html>"`
Construye un fragmento HTML donde se incluye directamente el valor del parámetro "name" dentro del cuerpo. Aquí está la vulnerabilidad porque no se hace ninguna validación ni escape.
14. `self.wfile.write(respuesta.encode('utf-8'))`
Envía la respuesta HTML codificada en UTF-8 al cliente que hizo la petición.
15. `if __name__ == "__main__":`
Verifica que el script se está ejecutando directamente, no importado como módulo.

16. `server = HTTPServer(('localhost', 8000), SimpleHandler)`

Crea una instancia de servidor HTTP que escuchará en el puerto 8000 y usará la clase SimpleHandler para procesar las peticiones.

17. `print("Servidor iniciado en http://localhost:8000")`

Muestra un mensaje informando que el servidor está activo y en qué URL.

18. `server.serve_forever()`

Inicia el servidor para que acepte peticiones indefinidamente hasta que se detenga manualmente.

Ejercicio 2-Python. (Respuesta al final de los códigos)

- Indica las vulnerabilidades que se han producido en estas partes de código, si se hubieran producido, poniendo ese trozo de código y diciendo qué tipo de vulnerabilidad se ha producido
- Indica las vulnerabilidades que se han salvado en estas partes de código, si se hubieran salvado, poniendo ese trozo de código y diciendo qué tipo de vulnerabilidad se ha salvado.

```
#!/usr/bin/env python3
```

```
# Importa módulo para operaciones del sistema de archivos
```

```
import os
```

```
# Importa módulo para ejecutar comandos del sistema
```

```
import subprocess
```

```
# Importa módulo para evaluar expresiones seguras
```

```
import ast
```

```
# Importa módulo para expresiones regulares
```

```
import re
```

```
# Define función que ejecuta comandos del usuario
```

```
def ejecutar_comando_usuario():
```

```
# PIDE al usuario introducir comando del sistema
```

```
comando = input("Introduce un comando del sistema: ")
```

```
# Define comandos permitidos
```

```
comandos_permitidos = ['ls', 'pwd', 'whoami', 'date']
```

```
# Extrae solo la primera palabra (nombre del comando)
```

```
cmd_base = comando.split()[0] if comando.split() else "
```

```
# Verifica si el comando está en la lista blanca
```

```
if cmd_base not in comandos_permitidos:
```

```
    return "Comando no permitido"
```

```
# Inicia bloque try para manejo de excepciones
```

```
try:
```

```

# Ejecuta comando

resultado = subprocess.run([cmd_base], capture_output=True, text=True, timeout=5)

# Devuelve resultado del comando ejecutado

return f" {resultado.stdout}"

# Captura excepción

except subprocess.TimeoutExpired:

    # Mensaje de error

    return "Comando excedió tiempo límite"


# Define función para subida de archivos

def subir_archivo():

    # PIDE al usuario nombre del archivo

    nombre_archivo = input("Nombre del archivo a subir: ")

    # PIDE al usuario contenido del archivo

    contenido = input("Contenido del archivo: ")

    # Se quiere poner delimitar el nombre y la longitud

    nombre_sanitizado = re.sub(r'^a-zA-Z0-9_-]', "", nombre_archivo)[:50]

    # Define directorio base permitido para uploads

    directorio_base = "/var/www/uploads/"

    # Construye dentro del directorio base

    ruta = os.path.realpath(os.path.join(directorio_base, nombre_sanitizado))

    # Verifica que la ruta esté dentro del directorio permitido

    if not ruta.startswith(os.path.realpath(directorio_base)):

        #

        return "Ruta no permitida"

    # Verifica y limita que el contenido no exceda 10KB

    if len(contenido) > 1024 * 10:

        # Rechaza archivos demasiado grandes

        return "Contenido demasiado grande (máx 10KB)"

    # Inicia bloque try para escritura de archivo

```

try:

Abre el archivo en modo escritura en la ruta validada

with open(ruta, 'w') **as** f:

Escribe el contenido proporcionado por el usuario

f.write(contenido)

Confirma guardado del archivo

return f'Archivo seguro guardado: {nombre_sanitizado}'

Captura error de permisos al escribir el archivo

except PermissionError:

Mensaje de error de permisos

return "Error de permisos"

Define función para evaluar expresiones matemáticas

def procesar_expresion():

PIDE al usuario una expresión matemática

expresion = input("Introduce una expresión matemática: ")

Inicia bloque try para evaluación segura

try:

Evalúa literales permitidos (números, strings, tuplas, listas, etc.)

resultado = ast.literal_eval(expresion)

Devuelve resultado de la expresión válida

return f' Resultado: {resultado}'

Captura errores de sintaxis o valores no válidos

except (ValueError, SyntaxError):

Rechaza expresiones que no sean literales seguros

return "Expresión no segura (solo números y strings simples)"

Respuesta:

En la primera función, hemos evitado un ataque que permita ejecutar cualquier tipo de código por no tener validación. Ayuda el:

```
if cmd_base not in comandos_permitidos:
```

```
    return "Comando no permitido"
```

Este comando no permite ejecutar código que no está en la lista de comandos permitidos. Sin embargo, no valida si el usuario tiene permisos para ese tipo de comandos. Aunque suponemos que se hace a él directamente. No tenemos una validación de que haga, por ejemplo, el "ls" a archivos al que tiene acceso solamente.

En la segunda función, aunque se verifique en esta línea la ruta y el tamaño del archivo:

```
if not ruta.startswith(os.path.realpath(directorio_base)):
```

```
    return "Ruta no permitida"
```

```
# Verifica y limita que el contenido no exceda 10KB
```

```
if len(contenido) > 1024 * 10:
```

```
    # Rechaza archivos demasiado grandes
```

```
    return "Contenido demasiado grande (máx 10KB)"
```

No validamos que el archivo sea legítimo. Es decir, podría ser un código malicioso lo que estamos abriendo. Se pide el contenido del archivo pero no se valida. Deberíamos validar también que el archivo subido sea legítimo y se puede abrir correctamente. Para evitar vulnerabilidades XSS o CSRF, debemos validar lo que el usuario quiere escribir dentro del archivo. Esta parte del código sería vulnerable:

```
try:
```

```
    # Abre el archivo en modo escritura en la ruta validada
```

```
    with open(ruta, 'w') as f:
```

```
        # Escribe el contenido proporcionado por el usuario
```

```
        f.write(contenido)
```

```
    # Confirma guardado del archivo
```

```
    return f"Archivo seguro guardado: {nombre_sanitizado}"
```

Porque escribe y guarda el archivo sin sanitizar lo que se ha escrito en contenidos. Deberíamos validar y escapar los caracteres escritos. Sino se podría ver como código malicioso y ser ejecutado por otro usuario. Por lo que se podría escapar o incluso sanitizar el contenido del archivo, al igual que hemos hecho con el nombre.

En el último código, se evalúa los literales permitidos gracias a las siguientes líneas del código:

try:

Evalúa literales permitidos (números, strings, tuplas, listas, etc.)

resultado = ast.literal_eval(expresion)

Devuelve resultado de la expresión válida

return f"Resultado: {resultado}"

Captura errores de sintaxis o valores no válidos

except (ValueError, SyntaxError):

Rechaza expresiones que no sean literales seguros

return "Expresión no segura (solo números y strings simples)"

Esto evita ataques relacionados con la mala validación de caracteres. Aunque en el error nos indica que solo se deben admitir números y strings simples, y en el `ast.literal_eval` evaluamos más permitidos como tuplas o listas. Si queremos solo strings simples y números yo usaría `"isinstance"` para comprobar que se `string`, `int` o `float`. Sin embargo, esto ya es más detallado. Tampoco validamos si la expresión proporcionada por el usuario tiene sentido. Puesto que el usuario podría haber puesto una expresión como `"3/0"` lo que daría error y podría dañar el programa dejándolo vulnerable. Además, se muestra el resultado sin verificar que sea correcto, es decir que no haya ocurrido nada extraño como podría ser un resultado de algo dividido entre cero.

También se puede ver un mal manejo de errores, pues se guardan todos los errores y se muestran por pantalla tal cual es. Esto podría ayudar a un atacante a identificar los errores y ver cómo explotarlos. Yo metería un mensaje generalizado para todos los errores. Aunque este no es gran problema en las funciones que hemos vistos, en funciones más complejas podrían afectar al programa entero. Lo que provocaría que el atacante pudiera explotar la vulnerabilidad de los errores. Las partes que implican esta vulnerabilidad son las excepciones, como en el último código:

Captura errores de sintaxis o valores no válidos

except (ValueError, SyntaxError):

Rechaza expresiones que no sean literales seguros

return "Expresión no segura (solo números y strings simples)"

Se especifica el tipo de error `"Value"` o `"Syntax"`. Aunque está bien hecho el mensaje con un simple `"Expresión no segura"`, es posible que se guarde en el log. Así que el usuario solo tendría que ver el log para ver que tipo de error es.